UNIVERSIDADE DO VALE DO RIO DOS SINOS (UNISINOS)

CURSO DE ENGENHARIA DA COMPUTAÇÃO

GUIA DE DOMÍNIO TÉCNICO E DEFESA DE BANCA

MÓDULO 2: Rede Neural Recorrente LSTM (Estágio 1 - Track A)

Portas Forget, Input e Output ⟶ Atualização da Célula ⟶ Janelamento Temporal ⟶ Inferência Recursiva Multi-Horizonte

---

Candidato: Andrio Gabriel Epping | **Papel:** Orientador Técnico & Examinador Rigoroso de Banca |
Data: Novembro de 2026

Apresentação do Módulo de Treinamento

Este guia constitui o material de domínio aprofundado do **Módulo 2: Rede Neural Recorrente LSTM (Long Short-Term Memory)**, correspondente ao Estágio 1 do pipeline do Track A do Trabalho de Conclusão de Curso (TCC).

O objetivo deste módulo é garantir que o candidato compreenda, com precisão matemática irrepreensível, como cada porta lógica regula o fluxo de gradientes e preserva a memória do óleo isolante, como o janelamento temporal contínuo foi implementado em Python sobre tensores tridimensionais, quais são as dimensões exatas de pesos e parâmetros treináveis, por que a comparação com a persistência ingênua esclarece o comportamento dos dados, e como responder com segurança inabalável às arguições mais rigorosas da banca examinadora.

Sumário

1 Pilar 1: Fundamentação Matemática e Teórica **3**
- 1.1 A Limitação das RNNs Simples e o Mecanismo de Portas da LSTM . . . 3
- 1.2 Equacionamento Algébrico das Portas e Atualização Celular . . . 3
- 1.3 Significado Físico de Cada Termo no Contexto de DGA . . . 4
- 1.4 Topologia Empilhada e Camada de Saída Linear . . . 4
- 1.5 Formulação Matemática do Janelamento e Inferência Recursiva . . . 5

2 Pilar 2: Rastreamento no Código (Linha a Linha) **6**
- 2.1 Mapeamento dos Arquivos do Módulo LSTM . . . 6
- 2.2 Rastreamento da Construção do Modelo (`src/lstm/model.py`) . . . 6
- 2.3 Cálculo Analítico do Número de Parâmetros Treináveis . . . 6
- 2.4 Rastreamento da Normalização `ScalerPerGas` (`src/lstm/data_loader.py`) . . . 7
- 2.5 Rastreamento da Inferência Recursiva (`build_katser_features.py`) . . . 7
- 2.6 Tabela de Evolução Dimensional dos Tensores . . . 8

3 Pilar 3: Decisões de Projeto e Nuances do Meu Trabalho **8**
- 3.1 Justificativa Arquitetural: Por que LSTM e não RNN, GRU ou Transformer? . . . 8
- 3.2 Seleção da Janela Ótima ($w^* = 50$) e Horizontes de Previsão ($h = 14, 28$) . . . 9
- 3.3 A Descoberta Crucial: LSTM vs. Baseline de Persistência . . . 9
- 3.4 Nuance Metodológica: O Risco de Viés no Treino do XGBoost . . . 10

4 Pilar 4: Sabatina da Banca (3 Perguntas Difíceis e Gabaritos) 10

1 Pilar 1: Fundamentação Matemática e Teórica

1.1 A Limitação das RNNs Simples e o Mecanismo de Portas da LSTM

As redes neurais recorrentes convencionais (Vanilla RNN) processam sequências temporais aplicando uma transformação estritamente não-linear recorrente a cada passo de tempo t:

$$\mathbf{h}_t = \tanh\left(\mathbf{W}_x \mathbf{x}_t + \mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{b}\right) \tag{1}$$

onde $\mathbf{x}_t \in \mathbb{R}^D$ é o vetor de entrada no instante t e $\mathbf{h}_t \in \mathbb{R}^H$ é o estado oculto.

Durante o treinamento pelo algoritmo de Retropropagação no Tempo (*Backpropagation Through Time* – BPTT), o gradiente da função de perda $\mathcal{L}$ em relação ao estado oculto inicial $\mathbf{h}_1$ envolve o produto encadeado da matriz Jacobiana:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_1} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_T} \prod_{k=2}^{T} \frac{\partial \mathbf{h}_k}{\partial \mathbf{h}_{k-1}} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_T} \prod_{k=2}^{T} \operatorname{diag}\left(1 - \tanh^2(\cdot)\right) \mathbf{W}_h^\top \tag{2}$$

Se o maior autovalor de $\mathbf{W}_h$ for inferior a 1, os autovalores do produto aproximam-se de zero exponencialmente com o aumento do horizonte temporal T ($\lambda^T \to 0$), provocando o **desaparecimento do gradiente** (*vanishing gradient*) e tornando a rede incapaz de aprender dependências temporais longas (superiores a 10–15 passos). Se for maior que 1, ocorre a explosão do gradiente.

A arquitetura *Long Short-Term Memory* (Hochreiter & Schmidhuber, 1997; Graves, 2013) soluciona essa deficiência ao introduzir um canal linear aditivo dedicado – o **Estado da Célula** ($\mathbf{C}_t \in \mathbb{R}^H$) – regulado por três mecanismos de portas não-lineares baseados na função sigmoide logísitica:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \in (0, 1) \tag{3}$$

1.2 Equacionamento Algébrico das Portas e Atualização Celular

A cada instante de tempo $t \in \{1, \ldots, w\}$ dentro da janela de entrada, a célula LSTM recebe o vetor de gases atual $\mathbf{x}_t \in \mathbb{R}^D$ ($D = 4$) e o vetor de estado oculto do passo anterior $\mathbf{h}_{t-1} \in \mathbb{R}^H$ ($H = 64$). O fluxo de informação obedece ao seguinte sistema de equações:

Sistema de Equações Fundamentais da Célula LSTM

1. Porta de Esquecimento (*Forget Gate*):

$$\mathbf{f}_t = \sigma\left(\mathbf{W}_f \mathbf{x}_t + \mathbf{U}_f \mathbf{h}_{t-1} + \mathbf{b}_f\right) \in (0, 1)^H \tag{4}$$

2. Porta de Entrada (*Input Gate*):

$$\mathbf{i}_t = \sigma\left(\mathbf{W}_i \mathbf{x}_t + \mathbf{U}_i \mathbf{h}_{t-1} + \mathbf{b}_i\right) \in (0, 1)^H \tag{5}$$

3. Vetor de Estado Candidato (*Candidate Cell State*):

$$\tilde{\mathbf{C}}_t = \tanh\left(\mathbf{W}_c \mathbf{x}_t + \mathbf{U}_c \mathbf{h}_{t-1} + \mathbf{b}_c\right) \in (-1, 1)^H \tag{6}$$

4. Atualização do Estado da Célula (*Cell State Update*):

$$\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t \in \mathbb{R}^H \tag{7}$$

5. Porta de Saída (*Output Gate*):

$$\mathbf{o}_t = \sigma\left(\mathbf{W}_o \mathbf{x}_t + \mathbf{U}_o \mathbf{h}_{t-1} + \mathbf{b}_o\right) \in (0, 1)^H \tag{8}$$

6. Estado Oculto Resultante (*Hidden State*):

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{C}_t) \in (-1, 1)^H \tag{9}$$

onde:

- $\mathbf{W}_f, \mathbf{W}_i, \mathbf{W}_c, \mathbf{W}_o \in \mathbb{R}^{H \times D}$ são as matrizes de pesos da entrada externa $\mathbf{x}_t$;
- $\mathbf{U}_f, \mathbf{U}_i, \mathbf{U}_c, \mathbf{U}_o \in \mathbb{R}^{H \times H}$ são as matrizes de pesos recorrentes do estado anterior $\mathbf{h}_{t-1}$;
- $\mathbf{b}_f, \mathbf{b}_i, \mathbf{b}_c, \mathbf{b}_o \in \mathbb{R}^{H}$ são os vetores de viés (*biases*);
- $\odot$ denota o produto de Hadamard (multiplicação elemento a elemento entre vetores).

1.3 Significado Físico de Cada Termo no Contexto de DGA

No monitoramento contínuo de transformadores de potência, cada componente da equação reflete fenômenos físico-químicos reais da decomposição de óleo mineral e papel isolante:

- **Porta de Esquecimento ($\mathbf{f}_t$):** Determina a fração da concentração histórica acumulada $\mathbf{C}_{t-1}$ a ser preservada. Quando $\mathbf{f}_t \approx \mathbf{1}$, o histórico térmico e químico é preservado integralmente ao longo de semanas. Quando $\mathbf{f}_t \to \mathbf{0}$, a rede realiza o descarte de memória, simulando intervenções de manutenção física do equipamento, tais como a desumidificação, desgaseificação ou recirculação forçada do óleo isolante na subestação.
- **Porta de Entrada ($\mathbf{i}_t$) e Candidato ($\tilde{\mathbf{C}}_t$):** O vetor $\tilde{\mathbf{C}}_t$ expressa a nova taxa instantânea de geração gasosa decorrente de um estresse elétrico ou térmico recente. A porta $\mathbf{i}_t$ atua como uma barreira seletiva: filtra variações ruidosas de baixa amplitude (flutuações circadianas de temperatura ambiente) e absorve prontamente degraus ou tendências aceleradas geradas por descargas elétricas incipientes.
- **Atualização Aditiva ($\mathbf{C}_t$):** A equação (7) modela a **conservação de massa e solubilidade** dos gases no óleo mineral. Como a derivada parcial:

$$\frac{\partial \mathbf{C}_t}{\partial \mathbf{C}_{t-1}} = \operatorname{diag}(\mathbf{f}_t) \tag{10}$$

  não envolve multiplicações diretas por matrizes de pesos saturadas, o gradiente propaga-se no carrossel de erro constante (*Constant Error Carousel* – CEC) sem atenuar exponencialmente quando $\mathbf{f}_t \approx 1$.
- **Porta de Saída ($\mathbf{o}_t$) e Estado Oculto ($\mathbf{h}_t$):** O estado da célula $\mathbf{C}_t$ retém a concentração física latente acumulada, enquanto $\mathbf{h}_t$ projeta apenas a fração necessária para prever a concentração superficial observável pelos sensores no passo imediatamente seguinte.

1.4 Topologia Empilhada e Camada de Saída Linear

A arquitetura implementada no TCC (`src/lstm/model.py`) emprega duas camadas LSTM empilhadas (*stacked*):

1. **Camada Recorrente 1:** Recebe a sequência temporal tridimensional $\mathbf{X} \in \mathbb{R}^{B \times w \times D}$ e opera com `return_sequences=True`:

$$\mathbf{H}^{(1)} = \left[\mathbf{h}_1^{(1)}, \mathbf{h}_2^{(1)}, \ldots, \mathbf{h}_w^{(1)}\right] \in \mathbb{R}^{B \times w \times 64} \tag{11}$$

2. **Camada de Regularização Dropout 1:** Zera aleatoriamente 20% das conexões durante o treino com taxa $p = 0{,}20$.
3. **Camada Recorrente 2:** Processa a sequência $\mathbf{H}^{(1)}$ e opera com `return_sequences=False`, descartando as etapas intermediárias e emitindo apenas o último estado oculto:

$$\mathbf{h}_w^{(2)} \in \mathbb{R}^{B \times 64} \tag{12}$$

4. **Camada de Regularização Dropout 2:** Taxa de abandono $p = 0{,}20$.

5. **Camada Densa de Projeção Linear (One-Step Forecast):**

$$\hat{\mathbf{x}}_{t+1} = \mathbf{W}_{\text{out}}\mathbf{h}_w^{(2)} + \mathbf{b}_{\text{out}} \in \mathbb{R}^{B\times 4} \tag{13}$$

onde $\mathbf{W}_{\text{out}} \in \mathbb{R}^{4\times 64}$ e $\mathbf{b}_{\text{out}} \in \mathbb{R}^4$. Como se trata de regressão contínua em concentração física, **nenhuma função de ativação não-linear é aplicada na saída** (função de ativação linear/identidade).

1.5 Formulação Matemática do Janelamento e Inferência Recursiva

Matemática do Janelamento e Projeção Multi-Passo

Seja uma série temporal de $T = 420$ leituras horárias (período de amostragem de 12 horas, totalizando 210 dias de monitoramento).

1. Janelamento Deslizante ($w = 50, h = 1$):
Para cada série individual, constrói-se o conjunto de pares entrada-alvo:

$$\mathcal{D}_{\text{janelas}} = \{(\mathbf{W}_k,\ \mathbf{y}_k)\}_{k=1}^{T-w-h+1} \tag{14}$$

onde o número de amostras geradas por série é:

$$N_{\text{amostras}} = 420 - 50 - 1 + 1 = 370 \text{ pares por transformador} \tag{15}$$

A janela de entrada $\mathbf{W}_k$ e o alvo $\mathbf{y}_k$ são dados por:

$$\mathbf{W}_k = \begin{bmatrix} \mathbf{x}_k^\top \\ \mathbf{x}_{k+1}^\top \\ \vdots \\ \mathbf{x}_{k+w-1}^\top \end{bmatrix} \in \mathbb{R}^{50\times 4}, \quad \mathbf{y}_k = \mathbf{x}_{k+w} \in \mathbb{R}^4 \tag{16}$$

2. Função de Perda (MSE):

$$\mathcal{L}_{\text{MSE}}(\Theta) = \frac{1}{B}\sum_{b=1}^{B}\frac{1}{D}\sum_{d=1}^{4}(y_{b,d} - \hat{y}_{b,d})^2 \tag{17}$$

3. Inferência Recursiva Multi-Horizonte (Auto-Regressão Roll-Out):
Para projetar concentrações a horizontes $h \in \{14, 28\}$ passos à frente (7 e 14 dias), o modelo é executado recursivamente h vezes:

$$\hat{\mathbf{x}}_{T+1} = f_\Theta\left([\mathbf{x}_{T-w+1}, \mathbf{x}_{T-w+2}, \ldots, \mathbf{x}_T]\right) \tag{18}$$
$$\hat{\mathbf{x}}_{T+2} = f_\Theta\left([\mathbf{x}_{T-w+2}, \ldots, \mathbf{x}_T, \hat{\mathbf{x}}_{T+1}]\right) \tag{19}$$
$$\vdots$$
$$\hat{\mathbf{x}}_{T+h} = f_\Theta\left([\mathbf{x}_{T-w+h}, \ldots, \hat{\mathbf{x}}_{T+h-1}]\right) \tag{20}$$

A cada iteração, a janela temporal descarta a observação mais antiga na posição 0 e anexa a estimativa prevista mais recente no final da fila (buffer circular / *rolling window*).

2 Pilar 2: Rastreamento no Código (Linha a Linha)

2.1 Mapeamento dos Arquivos do Módulo LSTM

Os arquivos que implementam a Fase 3 e o Estágio 1 do Track A estão localizados na pasta `src/lstm/` e `src/preprocessing/`:

- **`src/preprocessing/windowing.py`:** Funções `criar_janelas()` e `criar_dataset_janelado()`.
- **`src/lstm/data_loader.py`:** Classe `ScalerPerGas` e função `preparar_datasets()`.
- **`src/lstm/model.py`:** Funções `construir_modelo()` e `fixar_seeds()`.
- **`src/lstm/trainer.py`:** Funções `treinar_modelo()` e `buscar_melhor_janela()`.
- **`src/lstm/multi_horizon_eval.py`:** Função `previsao_recursiva()` e avaliação multi-horizonte.
- **`src/feature_engineering/build_katser_features.py`:** Função `prever_gases_futuros()` para extração das variáveis $h = 14$ e $h = 28$ do XGBoost.

2.2 Rastreamento da Construção do Modelo (`src/lstm/model.py`)

O código em Python que implementa a arquitetura de 2 camadas empilhadas com Dropout e saída Dense:

```python
def construir_modelo(w: int, n_features: int = 4):
    inputs = tf.keras.Input(shape=(w, n_features), name="gas_window")

    # Camada 1: return_sequences=True emite shape (batch, 50, 64)
    x = tf.keras.layers.LSTM(64, return_sequences=True, name="lstm_1")(inputs)
    x = tf.keras.layers.Dropout(0.2, name="dropout_1")(x)

    # Camada 2: return_sequences=False emite apenas o ultimo estado (batch, 64)
    x = tf.keras.layers.LSTM(64, return_sequences=False, name="lstm_2")(x)
    x = tf.keras.layers.Dropout(0.2, name="dropout_2")(x)

    # Saida: projecao linear sem ativacao para os 4 gases (batch, 4)
    outputs = tf.keras.layers.Dense(n_features, name="gas_forecast")(x)

    model = tf.keras.Model(inputs=inputs, outputs=outputs,
        name="lstm_dga_forecast")
    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
        loss="mse",
        metrics=["mae"]
    )
    return model
```

2.3 Cálculo Analítico do Número de Parâmetros Treináveis

Um questionamento frequente em bancas de Engenharia é exigir a demonstração exata de quantos pesos existem na rede:

Dedução Formal dos Pesos Treináveis da Rede LSTM

Uma camada LSTM com dimensão de entrada D e dimensão oculta H possui 4 blocos de transformação afim $(\mathbf{f}_t, \mathbf{i}_t, \tilde{\mathbf{C}}_t, \mathbf{o}_t)$. O número de parâmetros por camada é:

$$P_{\text{LSTM}} = 4 \times (H \times D + H \times H + H) = 4 \times H \times (D + H + 1) \tag{21}$$

1. **Camada `lstm_1` ($D = 4, H = 64$):**

$$P_1 = 4\times(64 \times 4 + 64 \times 64 + 64) = 4\times(256+4096+64) = 4\times4416 = \mathbf{17.664} \text{ parâmetros} \quad (22)$$

2. **Camada `lstm_2` ($D = 64, H = 64$):**

$$P_2 = 4 \times (64 \times 64 + 64 \times 64 + 64) = 4 \times (4096 + 4096 + 64) = 4 \times 8256 = \mathbf{33.024} \text{ parâmetros} \quad (23)$$

3. **Camada `gas_forecast` (Densa com entrada 64 e saída 4):**

$$P_3 = (64 \times 4) + 4 = 256 + 4 = \mathbf{260} \text{ parâmetros} \quad (24)$$

Total Treinável da Rede:

$$P_{\text{total}} = 17.664 + 33.024 + 260 = \mathbf{50.948} \text{ parâmetros treináveis} \quad (25)$$

As camadas de `Dropout` não possuem nenhum parâmetro treinável ($P = 0$).

2.4 Rastreamento da Normalização `ScalerPerGas` (`src/lstm/data_loader.py`)

A normalização independente por gás equaliza as amplitudes discrepantes:

```python
class ScalerPerGas:
    def fit(self, series_treino: np.ndarray) -> "ScalerPerGas":
        # Achatamento (n_series * T, n_gases) mantendo integridade temporal
        flat = series_treino.reshape(-1, series_treino.shape[-1])  # (1785*420, 4)
        self.min_ = flat.min(axis=0)                               # Shape: (4,)
        self.scale_ = flat.max(axis=0) - flat.min(axis=0)          # Shape: (4,)
        self.scale_ = np.where(self.scale_ == 0, 1.0, self.scale_)
        return self

    def transform(self, series: np.ndarray) -> np.ndarray:
        return (series - self.min_) / self.scale_  # Normaliza estritamente em [0, 1]

    def inverse_transform(self, series: np.ndarray) -> np.ndarray:
        return series * self.scale_ + self.min_    # Reverte para ppm ou escala original
```

2.5 Rastreamento da Inferência Recursiva (`build_katser_features.py`)

O laço auto-regressivo que projeta as variáveis $h = 14$ e $h = 28$ consumidas pelo XGBoost:

```python
def prever_gases_futuros(modelo, janela_norm: np.ndarray, h: int) -> np.ndarray:
    janela_atual = janela_norm.copy()  # Shape: (50, 4)

    for _ in range(h):
        # Cria dimensao de batch para inferencia no Keras: (1, 50, 4)
        x_input = janela_atual[np.newaxis, :, :]
        proximo = modelo.predict(x_input, verbose=0)[0]  # Shape: (4,)
        # Desloca a janela: elimina o passo mais antigo e anexa a predicao no fim
        janela_atual = np.vstack([janela_atual[1:], proximo])

    return proximo  # Retorna exclusivamente o vetor no horizonte h: shape (4,)
```

2.6 Tabela de Evolução Dimensional dos Tensores

A Tabela 1 sumariza as transformações de dimensões desde as séries brutas até a saída das features tabulares.

Tabela 1: Evolução Dimensional dos Tensores no Pipeline da LSTM.

Etapa / Tensor	Operação no Código	Shape do Tensor	Descrição Física
`s_treino`	`carregar_splits()`	$(1785, 420, 4)$	1.785 séries de treino, 420 horas, 4 gases
`s_teste`	`carregar_splits()`	$(900, 420, 4)$	900 séries oficiais de teste independente
`X_train`	`criar_dataset_janelado()`	$(660450, 50, 4)$	Janelas de 50 passos deslizantes
`y_train`	`criar_dataset_janelado()`	$(660450, 4)$	Próximo passo observável ($h = 1$)
`X_test`	`criar_dataset_janelado()`	$(333000, 50, 4)$	Janelas de teste para validação
`lstm_1`	Saída da 1ª camada LSTM	$(B, 50, 64)$	Sequência temporal de estados ocultos
`lstm_2`	Saída da 2ª camada LSTM	$(B, 64)$	Vetor final de estado latente
`outputs`	Saída da camada Densa	$(B, 4)$	Previsão pontual dos 4 gases ($h = 1$)
`previsao_raw`	`prever_gases_futuros()`	$(4,)$	Predição recursiva desnormalizada em h
`grupo3`	Concatenação $h = 14$ e $h = 28$	$(8,)$	8 atributos temporais para o XGBoost

3 Pilar 3: Decisões de Projeto e Nuances do Meu Trabalho

3.1 Justificativa Arquitetural: Por que LSTM e não RNN, GRU ou Transformer?

Por que a LSTM foi a arquitetura escolhida para a previsão temporal?

1. **Frente à RNN Simples:** Conforme demonstrado matematicamente na Seção 1.1, a RNN convencional sofre com o desaparecimento do gradiente para sequências de 50 passos, falhando em modelar a inércia térmica que dura semanas.

2. **Frente à GRU (*Gated Recurrent Unit*):** A GRU funde o estado da célula $\mathbf{C}_t$ com o estado oculto $\mathbf{h}_t$, utilizando apenas duas portas (Update e Reset). Embora a GRU tenha menos parâmetros (≈ 38.000 contra 50.948), o desacoplamento explícito da LSTM entre memória de longo prazo ($\mathbf{C}_t$, estoque acumulado de óleo) e ativação de curto prazo ($\mathbf{h}_t$, leitura imediata de superfície) oferece uma representação mais aderente à física de difusão de gases em fluidos térmicos.

3. **Frente a Transformers (como Informer / Autoformer):** Redes baseadas em mecanismos de atenção (*Self-Attention*) exigem dezenas de milhares de amostras heterogêneas para calibrar matrizes de projeção de Query, Key e Value sem sobreajuste. Na literatura de DGA aberta, a base Katser possui 2.100 transformadores em condições padronizadas; o custo computacional de um Transformer não se justifica para séries que possuem dinâmica linear de alta inércia, como demonstrado na Tabela 2 do TCC.

3.2 Seleção da Janela Ótima ($w^* = 50$) e Horizontes de Previsão ($h = 14, 28$)

Critérios de Projeto da Janela Temporal e Horizontes

- **Seleção da Janela Histórica ($w = 50$):** Na Fase 3, testaram-se quatro tamanhos candidatos na partição de validação: $w \in \{10, 20, 30, 50\}$. Janelas curtas ($w = 10$, 5 dias) mostraram-se sensíveis a ruídos passageiros de sensores online. A janela de $w^* = 50$ (correspondente a 25 dias de operação contínua) obteve o menor MAE médio de validação, capturando com precisão a taxa contínua de decomposição molecular sem introduzir atraso dinâmico indesejado.
- **Definição dos Horizontes ($h = 14$ e $h = 28$):** Como cada passo temporal equivale a 12 horas:
  - $h = 14$ passos $\implies 14 \times 12\text{h} = 168$ horas $=$ **7** dias (1 semana), que corresponde à rotina operacional de inspeção semanal em subestações.
  - $h = 28$ passos $\implies 28 \times 12\text{h} = 336$ horas $=$ **14** dias (2 semanas), prazo suficiente para agendamento de manutenção preditiva sem parada intempestiva da linha.

3.3 A Descoberta Crucial: LSTM vs. Baseline de Persistência

A Tabela 2 compara a acurácia preditiva da LSTM com a persistência ingênua ($\hat{\mathbf{x}}_{t+h} = \mathbf{x}_t$), demonstrando por que a hipótese de ganho preditivo no diagnóstico foi rejeitada no Track A.

Tabela 2: Comparação entre a LSTM Recursiva e o Baseline de Persistência Ingênua.

Horizonte (h)	**Tempo Real**	**R^2 Persistência**	**R^2 LSTM**	**MAE% LSTM**	**Diagnóstico Técnico**
$h = 1$	12 horas	1,0000	0,9996	0,90%	Estacionariedade quase perfeita
$h = 7$	3,5 dias	0,9992	0,9991	1,31%	Empate técnico
$h = 14$	7 dias	0,9970	0,9979	2,05%	Ganho marginal da LSTM ($\Delta R^2 = +0,0009$)
$h = 28$	14 dias	0,9886	0,9928	3,76%	Ganho de $\Delta R^2 = +0,0042$
$h = 98$	7 semanas	0,8940	0,8846	14,96%	Limite de confiabilidade ($R^2_{\text{H}_2} = 0,8483 \geq 0,80$)
$h = 112$	8 semanas	0,8696	0,8380	17,68%	Rompimento do critério ($R^2_{\text{H}_2} = 0,7877 < 0,80$)

Conclusão de Engenharia:
A dinâmica dos transformadores na base Katser apresenta uma inércia temporal tão elevada que a repetição pura e simples da leitura de hoje já garante $R^2 = 0,9970$ após 7 dias de operação. Essa constatação explica matematicamente por que o classificador XGBoost atinge praticamente o mesmo F_1-score com ou sem as variáveis da LSTM (0,7604 vs 0,7602).

3.4 Nuance Metodológica: O Risco de Viés no Treino do XGBoost

Ponto Crítico: Previsões In-Sample no Treino vs. Out-of-Sample no Teste

A Nuance Técnica Identificada:
No pipeline do Track A, para construir o arquivo `katser_features_train.csv`, a LSTM treinada gerou as variáveis `lstm_h14_*` e `lstm_h28_*` para as 1.785 séries que a própria LSTM observou durante o seu treinamento (previsões *in-sample*). Por outro lado, para o arquivo `katser_features_test.csv`, as previsões foram geradas sobre as 900 séries de teste que a LSTM nunca viu (*out-of-sample*).

Impacto e Mitigação:
Isso poderia gerar um viés favorável ao treino, onde as previsões seriam mais precisas no treino do que no teste. Contudo, como o erro da LSTM no teste permaneceu minúsculo ($R^2 = 0{,}9979$ em $h = 14$), essa discrepância foi praticamente nula na prática. O TCC declara essa limitação de forma transparente nas Seções 3.6.1 e 5.3, reconhecendo que o treinamento híbrido não utilizou um esquema estrito de validação cruzada *out-of-fold* para a geração de features na base de desenvolvimento.

4 Pilar 4: Sabatina da Banca (3 Perguntas Difíceis e Gabaritos)

Pergunta Crítica da Banca: 1. Dedução de Pesos e Operações Matriciais da LSTM

"O seu trabalho utiliza uma rede LSTM com duas camadas empilhadas de 64 unidades cada, afirmando que o modelo totaliza 50.948 parâmetros treináveis. Como esses parâmetros são alocados exatamente entre as matrizes de entrada, matrizes recorrentes e vetores de bias de cada porta lógica? Mostre analiticamente a dedução desse número e explique como bibliotecas otimizadas como TensorFlow/cuDNN empacotam essas operações para execução eficiente na GPU."

Gabarito Técnico Ideal de Resposta (Blindagem de Banca)

Estratégia de Resposta:
Demonstre domínio analítico irrefutável deduzindo as equações de parâmetros no quadro ou com firmeza verbal:

1. **Fórmula Geral de Parâmetros por Célula LSTM:** "Uma camada LSTM possui 4 portas funcionais: porta de esquecimento ($\mathbf{f}$), porta de entrada ($\mathbf{i}$), vetor candidato ($\tilde{\mathbf{C}}$) e porta de saída ($\mathbf{o}$). Cada porta possui sua própria matriz de pesos para a entrada externa ($\mathbf{W} \in \mathbb{R}^{H \times D}$), sua matriz de pesos para a realimentação do estado oculto ($\mathbf{U} \in \mathbb{R}^{H \times H}$) e seu vetor de viés ($\mathbf{b} \in \mathbb{R}^{H}$). Portanto, a equação geral é:

$$P_{\text{LSTM}} = 4 \times \Big(H \cdot D + H^2 + H \Big) = 4 \cdot H \cdot (D + H + 1) \tag{26}$$

2. **Cálculo da Primeira Camada (`lstm_1`, $D = 4, H = 64$):**

$$P_1 = 4 \times 64 \times (4 + 64 + 1) = 256 \times 69 = \mathbf{17.664} \text{ parâmetros} \tag{27}$$

3. **Cálculo da Segunda Camada (`lstm_2`, $D = 64, H = 64$):**

$$P_2 = 4 \times 64 \times (64 + 64 + 1) = 256 \times 129 = \mathbf{33.024} \text{ parâmetros} \tag{28}$$

4. **Cálculo da Camada Densa Final (Entrada 64, Saída 4):**

$$P_3 = (64 \times 4) + 4 = 256 + 4 = \mathbf{260} \text{ parâmetros} \tag{29}$$

   Somando os blocos: $17.664 + 33.024 + 260 = \mathbf{50.948}$ parâmetros treináveis exatos."

5. **Empacotamento Computacional no TensorFlow / cuDNN:** "Para maximizar a vazão em hardware paralelo (GPU), o TensorFlow não executa 4 multiplicações de matrizes isoladas. Ele empacota as matrizes de entrada em uma única matriz combinada $\mathbf{W} \in \mathbb{R}^{4H \times D}$ e as matrizes recorrentes em $\mathbf{U} \in \mathbb{R}^{4H \times H}$. Uma única instrução GEMM (General Matrix Multiply) computa $\mathbf{Z}_t = \mathbf{W}\mathbf{x}_t + \mathbf{U}\mathbf{h}_{t-1} + \mathbf{b}$, e o vetor resultante de dimensão $4H = 256$ é fatiado (*sliced*) em 4 vetores de dimensão 64 para aplicação simultânea das funções sigmoide e tangente hiperbólica."

Pergunta Crítica da Banca: 2. A Crítica da Persistência: A LSTM é Desnecessária?

"Na Tabela 2 do seu TCC, os dados mostram que a persistência ingênua ($\hat{y}_{t+h} = y_t$) atinge um coeficiente de determinação $R^2 = 0{,}9970$ no horizonte de 7 dias ($h = 14$), enquanto a sua LSTM complexa de 50 mil parâmetros atinge $R^2 = 0{,}9979$. A diferença é de menos de 0,1%! E em 7 semanas ($h = 98$), a persistência chega a ser superior à LSTM ($0{,}8940$ vs $0{,}8846$). Diante desses números, a sua rede neural não é um modelo desnecessariamente complexo para dados que quase não variam no tempo?"

Gabarito Técnico Ideal de Resposta (Blindagem de Banca)

Estratégia de Resposta:
Transforme o que parece ser uma fragilidade em uma demonstração de excelência científica, honestidade metodológica e domínio do processo físico.

1. **Reconhecimento da Inércia Física do Processo:** "O examinador toca no ponto nevrálgico do Track A. A persistência ingênua atinge $R^2 = 0{,}9970$ porque a dissolução de gases no óleo mineral sob regime estável é governada por processos de difusão molecular lentos, descritos pelas leis de Fick e termodinâmica de fluidos, onde a taxa diária de acúmulo em ppm é pequena em relação ao volume de centenas de litros de óleo do transformador."

2. **Onde a LSTM Agrega Valor:** "Embora na média global a persistência seja competitiva em razão da inércia, a persistência possui **viés estrutural zero de antecipação**: ela é matematicamente incapaz de prever acelerações de tendência, inflexões na taxa de geração de gás ou dinâmicas causadas pelo ciclo de aquecimento térmico do transformador. A LSTM captura derivadas de segunda ordem da concentração que um baseline constante não consegue antecipar."

3. **O Rigor Metodológico do Trabalho:** "O maior valor científico do nosso TCC foi justamente ter implementado o baseline de persistência e exposto esse confronto com total transparência. Grande parte dos artigos da literatura recente de Deep Learning aplicado a DGA publica modelos LSTM anunciando acurácias de 99% sem nunca compará-los contra a persistência ingênua. Nosso trabalho desmascarou essa ilusão: provamos que o $R^2 = 0{,}99$ da literatura decorre da estacionariedade das bases disponíveis, e não necessariamente da mágica do modelo recorrente. Ao demonstrar via teste de ablação que o acoplamento temporal não altera o diagnóstico final ($F_1 \approx 0{,}76$ em ambos os casos), entregamos uma contribuição metodológica genuína para a engenharia."

Pergunta Crítica da Banca: 3. Acúmulo de Erro Recursivo vs. Arquiteturas Diretas ou Seq2Seq

"Você utilizou uma estratégia auto-regressiva recursiva (alimentando a previsão do passo $t+1$ como entrada para o passo $t+2$ até $h = 112$), em vez de utilizar uma abordagem Direta (treinando um modelo independente para cada horizonte h) ou uma arquitetura Sequence-to-Sequence (Seq2Seq / Encoder-Decoder). Quais são as desvantagens teóricas do acúmulo de erros recursivos e por que você adotou essa estratégia no TCC?"

Gabarito Técnico Ideal de Resposta (Blindagem de Banca)

Estratégia de Resposta:
Apresente a fundamentação teórica de séries temporais sobre as três abordagens de multi-step forecasting e justifique a escolha com base no custo de treinamento e consistência dinâmica.

1. **A Patologia Teórica da Abordagem Recursiva:** "A abordagem recursiva sofre de **propagação e acumulação de erros estocásticos**:

$$\hat{\mathbf{x}}_{t+k} = f_{\Theta}\left(\hat{\mathbf{x}}_{t+k-1}\right) + \boldsymbol{\epsilon}_k \tag{30}$$

   Como o modelo foi treinado exclusivamente com pares (X, y) de dados reais limpos ($h = 1$), ao ser realimentado com suas próprias predições imperfeitas durante a inferência, ele opera sobre uma distribuição fora do domínio de treinamento (*exposure bias* / covariância de deslocamento). Isso explica exatamente por que, na Tabela 2, o R^2 do hidrogênio decai progressivamente de 0,9995 em $h = 1$ para 0,7877 em $h = 112$."

2. **Por que a Abordagem Direta Foi Rejeitada:** "A estratégia direta exigiria treinar K modelos neurais completamente independentes para cada horizonte de interesse $(f_{\Theta_1}, f_{\Theta_2}, \ldots, f_{\Theta_{112}})$. Para 12 horizontes, isso aumentaria o custo computacional de treinamento em 12 vezes ($12 \times 50.948 \approx 611.000$ parâmetros), além de quebrar a consistência física temporal entre passos adjacentes (o modelo $h = 15$ poderia prever um valor fisicamente incompatível com o modelo $h = 14$)."

3. **Por que a Abordagem Recursiva é a Mais Adequada para o Problema:** "Na operação de subestações de energia, o operador precisa de um modelo compacto e flexível que possa ser consultado para qualquer horizonte arbitrário (7 dias, 10 dias, 14 dias ou 30 dias) sem necessidade de re-treinamento de novos modelos. Além disso, a auto-regressão garante que a trajetória respeite a dinâmica contínua do sistema termodinâmico. O trabalho comprovou que a acumulação de erro permanece dentro dos limites operacionais de confiabilidade ($R^2 \geq 0{,}80$) até 7 semanas contínuas ($h = 98$), validando empiricamente a viabilidade da técnica para planejamento de manutenção preditiva."

Fim do Módulo 2 – Domínio Pleno da Rede LSTM para a Defesa de TCC!